Loss-Aware Tensor Factorisation for Large Language Models

Taniya Sarkar

April 28, 2026

1 From Optimal Brain Decomposition to Tensor-Aware LLM Compression

1.1 Foundations: Loss-Aware Low-Rank Decomposition

Low-rank decomposition is a fundamental tool for compressing large neural networks. Given a weight matrix $W \in \mathbb{R}^{m\times n}$, standard approaches approximate it by a rank-r factorization

$$W \approx BA, \quad B \in \mathbb{R}^{m\times r}, \quad A \in \mathbb{R}^{r\times n}, \quad r \ll \min(m,n),$$

typically obtained via singular value decomposition (SVD). This approximation minimizes the Frobenius norm error $\|W - \widehat{W}\|_F^2$, but does not account for the impact of the approximation on the model's prediction loss.

The key limitation of such approaches is that *not all directions in parameter space are equally important*. Perturbations in some directions may have negligible effect on the loss, while others may be highly sensitive. Therefore, an optimal compression method should be *loss-aware* rather than purely reconstruction-based.

The Optimal Brain Decomposition (OBD) framework addresses this issue by approximating the increase in task loss using second-order information (LeCun et al., 1989). Let $w = \mathrm{vec}(W)$ and $\widehat{w} = \mathrm{vec}(\widehat{W})$. The change in loss due to approximation is locally approximated as

$$\Delta L(w) \approx (w - \widehat{w})^\top H (w - \widehat{w}),$$

where H is the Hessian of the loss function with respect to the parameters.

Thus, instead of minimizing a Frobenius norm, OBD minimizes a *curvature-weighted error*, giving higher penalty to directions that strongly affect the loss.

1.2 Practical Approximation via Kronecker Factorization

Direct computation of the Hessian is infeasible for large models. To overcome this, OBD-LLM (Li et al., 2026) adopts a Kronecker-factored approximation:

$$H \approx C_x \otimes C_g,$$

where

$$C_x = \mathbb{E}[XX^\top], \qquad C_g = \mathbb{E}[GG^\top],$$

represent input activation covariance and output gradient covariance, respectively.

This leads to a key transformation:

$$\widetilde{W} = L_g^\top W L_x,$$

where $C_x = L_x L_x^\top$ and $C_g = L_g L_g^\top$ are Cholesky decompositions. SVD is then applied to the whitened matrix $\widetilde{W}$, yielding a loss-aware low-rank approximation. Finally, the factors are mapped back to the original space.

The central idea is:

Perform low-rank decomposition in a coordinate system where directions are scaled according to their importance for the task loss.

Empirically, this approach improves both perplexity and downstream accuracy compared to prior methods that only use input information.

1.3 Limitation: Matrix-Wise Decomposition

Despite its effectiveness, OBD-LLM operates at the level of individual matrices. However, transformer architectures exhibit strong structural redundancy across:

- layers,
- attention heads,
- projection types (e.g., Q, K, V, O).

Treating each matrix independently ignores this multiway structure.

1.4 Tensor View of Transformer Parameters

To exploit shared structure, we consider stacking related weight matrices into tensors. Bershatsky et al. (2024) uses this idea to extend the idea of LORA (Hu et al., 2021) to tensors. Standard LoRA applies independent low-rank updates:

$$\Delta W_\ell = A_\ell B_\ell^\top.$$

A tensor-aware extension shares structure across layers:

$$\Delta W_\ell = A S_\ell B^\top,$$

where A and B are shared, and S_ℓ captures layer-specific variation. For example, LOTR reduces parameters from $O(Ldr)$ to $O(dr + Lr^2)$ while enabling cross-layer information sharing. We can stack query projection matrices across layers:

$$\mathcal{W} \in \mathbb{R}^{L \times d_{\text{out}} \times d_{\text{in}}}, \qquad \mathcal{W}_{\ell,:,:} = W_\ell.$$

Including attention heads:

$$\mathcal{W} \in \mathbb{R}^{L \times H \times d_h \times d_h}.$$

More generally:

$$\mathcal{W} \in \mathbb{R}^{P \times L \times H \times d_h \times d_h},$$

where P indexes projection types.

This representation captures:

- cross-layer similarity,
- head-wise structure,
- shared patterns across modules.

1.5 Proposed Extension: Tensor-OBD

We extend OBD to tensors, yielding a framework we refer to as *Tensor-OBD*. The goal is to perform low-rank approximation in a *multiway loss-aware metric*.

Step 1: Tensor grouping. Select a set of related matrices and form a tensor $\mathcal{W}$.

Step 2: Curvature estimation. Estimate curvature factors along each mode. For example:

$$H \approx C^{(L)} \otimes C^{(\text{out})} \otimes C^{(\text{in})},$$

or with heads:

$$H \approx C^{(L)} \otimes C^{(H)} \otimes C^{(\text{out})} \otimes C^{(\text{in})}.$$

Step 3: Multiway whitening. Define the whitened tensor:

$$\widetilde{\mathcal{W}} = \mathcal{W} \times_1 (C^{(L)})^{1/2} \times_2 (C^{(\text{out})})^{1/2} \times_3 (C^{(\text{in})})^{1/2},$$

where $\times_k$ denotes the mode-k product.

Step 4: Tensor decomposition. Apply a tensor factorization such as:

- Tucker decomposition,
- CP decomposition,
- tensor-train (TT),
- shared-basis LoRA.

For instance, Tucker decomposition:

$$\mathcal{W} \approx \mathcal{G} \times_1 U^{(L)} \times_2 U^{(\text{out})} \times_3 U^{(\text{in})}.$$

Step 5: Unwhitening. Map factors back to the original parameter space.

1.6 Advantages of Tensor-OBD

The proposed framework offers several advantages:

1. Improved approximation quality. Combines loss-awareness (OBD) with structural sharing (tensor methods).

2. Parameter efficiency. Reduces redundancy across layers and heads.

3. Better initialization for PEFT. Provides principled initialization for low-rank adapters.

4. Unified view. Bridges compression and fine-tuning as structured approximation problems.

5. Scalability. Maintains efficiency using Kronecker-based curvature approximations.

1.7 Proposed Experimental Setup

To validate the approach, we propose the following comparisons:

- Layerwise SVD,
- OBD-LLM (matrix-wise),
- Tensor decomposition (loss-oblivious),
- Proposed Tensor-OBD.

Evaluation metrics include:

- Perplexity,
- Downstream accuracy,
- Parameter count,
- Compression ratio,
- Inference latency.

1.8 Conclusion

OBD-LLM demonstrates that low-rank approximation should be *loss-aware*. However, modern neural networks also exhibit strong *multiway structure* across layers and components. The proposed Tensor-OBD framework combines these two principles:

Loss-aware, multiway, shared low-rank approximation for transformer models.

By incorporating curvature information across multiple modes, Tensor-OBD has the potential to significantly improve both compression and parameter-efficient fine-tuning of large language models.

References

David Bershatsky et al. Lotr: Low tensor rank weight adaptation. *arXiv preprint arXiv:2402.01376*, 2024.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Yann LeCun, John Denker, and Sara Solla. Optimal brain damage. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1989.

Y. Li et al. Optimal brain decomposition for accurate llm low-rank approximation. *arXiv preprint arXiv:2604.00821*, 2026.